

Abstraction Project Notes

Brandon Tang

Symbolic Simulation for Functional Properties

The input to the symbolic simulation consists of an antecedent and a list of output constraints.

- The antecedent maps BDD variables to input wires in the circuit
- The output constraints map BDD expressions to signals in the circuit
 - These allow us to assert if a certain wires should be high/low/either at a certain time step for given conditions based on the antecedent
- These were traditionally written in the form of LTL formulae, but have been rewritten into a 4 tuple form in the 2013 paper, in VOSS and in JasperGold.

In JasperGold, antecedents and output constraints are written in the following form:

$$[(\text{signal}, \text{tickStart} : \text{tickEnd}, \text{highExpr}, \text{lowExpr})]$$

This is equivalent to the LTL formula $N^{\text{tick}} ((\text{highExpr} \rightarrow (\text{signal is 1}) \text{ and } \text{lowExpr} \rightarrow (\text{signal is 0})))$ for each tick in the range.

For an antecedent, this means that we will set the signal to be true if the highExpr is true and set the signal to be false if the lowExpr is true. Otherwise the signal is X.

For output constraints this means that when the assignment of BDD variables make highExpr true, we need the signal to be high from tickStart to tickEnd. Similarly for lowExpr. Thus each output constraint tuple can be split into 2 parts: the positive constraint where we need the signal to be true under a certain condition, and the negative constraint where we need the signal to be false under a certain condition. We let the highExpr and lowExpr be termed as the guards of the constraint.

Indexing Relations

Indexing relations are used to represent abstractions of circuit inputs. This means that they merge sets of circuit inputs into cases.

Indexing relations $R[X, C, T]$ are boolean formulae over BDD variables where

- X are the indexing BDD variables that perform the symbolic indexing
- T are the target BDD variables that are indexed. These correspond to input wires in the circuit
- C are the symbolic constants that correspond to target wires that should not be indexed

The indexing relation can be interpreted as follows:

- For each X, C , we cover the cases where $\exists T R[X, C, T]$.
- Since the indexing relation can be a many to many relation, we can have multiple T, C cases covered by a single X, C case indexing.

Preimage Operations

We define some operations involving the indexing relation here:

Suppose

- $P[C, T]$ represents some set of input cases (e.g. (C, T) is in the set iff $P[C, T]$ is true)
- $R[X, C, T]$ is an indexing relation

The **weak preimage** of P under R ,

$$P_R[X, C] = \exists T (R[X, C, T] \wedge P[C, T])$$

is the set of (X, C) that cover at least one (T, C) case in P

The **strong preimage** of P under R ,

$$P^R[X, C] = P_R \wedge \neg \exists T (R[X, C, T] \wedge \neg P[C, T])$$

is the set of (X, C) that cover at least one case in P and don't cover any cases not in P .

Image Operation

Taking the image of $H[X, C]$ returns the cases that are indexed by $H[X, C]$ under R .

$$\text{im}(H, R)[C, T] = \exists X (R[X, C, T] \wedge H[X, C])$$

While this is not directly used for finding proofs, it is useful for extracting counter examples for disproven properties.

Relationship between Preimages

For any fixed R and any given P and (X, C) we have one of the following cases:

- X, C indexes cases only in P
- X, C indexes cases only in $\neg P$
- X, C indexes cases in both P and $\neg P$
- X, C indexes cases in neither P nor $\neg P$

Any X, C that fall into the 4th category must correspond to empty indexing cases, i.e. don't map onto any (C, T) at all. Notice that this is independent of P , thus, we term the union of the first 3 cases as the domain of R .

$$\text{dom}(R)[X, C] = \exists T (R[X, C, T])$$

Observe that

- The weak preimage is the set of X, C that fall into the 1st and 3rd categories.
- The strong preimage is the set of X, C that fall into the 1st category.

We can note that the following are then also equivalent definitions of the strong preimage operation:

$$\begin{aligned} P^R[X, C] &= \text{dom}(R) \wedge \overline{\overline{P^R}} \\ &= \text{dom}(R) \wedge (\forall T (R[X, C, T] \rightarrow P[C, T])) \end{aligned}$$

Indexing Coverage Condition

For each property that we wish to check, we let the high and low expressions be P_i and Q_i respectively. The total space of inputs we need to check is then $B[C, T] = \bigvee_i (P_i \vee Q_i)$. In order for our abstraction to properly cover the space of required inputs, we require that

$$\forall T \forall C (B[C, T] \rightarrow \exists X (R[X, C, T]))$$

Generally we will be having guards that are equivalent to true, i.e. should hold for all inputs. In this case, $B[C, T] \equiv \text{True}$ so we coverage condition takes the simpler form:

$$\forall T \forall C \exists X (R[X, C, T])$$

Indexing Transformation and Symbolic Simulation Procedure

Suppose we are given an indexing relation, `idx_rel`, an antecedent list and an output constraint list. We will describe the procedure to prove that the output constraints hold.

We first apply the strong preimage operations on the high and low expressions of the antecedent tuples as follows:

```
antv = [(signal, tickStart:tickEnd, strongPreimage idx_rel highExpr, strongPreimage
idx_rel lowExpr)]
```

Next we run the symbolic simulator with the transformed antecedent. This produces an evaluation sequence which will contain sigHighExpr and sigLowExpr BDD expressions for when a signal at a certain time step will definitely be high or low. If for a given assignment, neither of these expressions are true, then the signal is unknown.

From this evaluation sequence, we can then check the output constraints. Suppose want to check a output constraint of the form (signal, tickStart:tickEnd, P, Q) where $P[C, T]$ and $Q[C, T]$ are the BDD expressions that correspond to inputs on which a signal should be high and low respectively. We will transform the output constraint to (signal, tickStart:tickEnd, P_R , Q_R) where P_R and Q_R are the weak preimages of P and Q under the indexing relation.

Then we analyse the highexpr and lowexpr BDD expressions from the evaluation sequence for the signal at each of the ticks in the tick range. Suppose that at a given tick t , for the property wire we have some the high expression as $H[X, C]$ and the low expression as $L[X, C]$. We term the high expression of the signal as the “residual”.

We will check if $(P_R \rightarrow H)[X, C] \equiv \text{True}$ and whether $(Q_R \rightarrow L)[X, C] \equiv \text{True}$. If both of these hold, then the output constraint is satisfied.

If we have $(P^R \rightarrow L) \neq \text{False}$ it means that some indexing cases that only index into P actually cause s to be low, this is a counter example to the property so the positive part of property is disproven. We can inspect the cases indexed by $P^R \rightarrow L$ to get the actual counter example in terms of the original circuit inputs.

Similarly, if we have $(Q^R \rightarrow H) \neq \text{False}$, we have disproven the negative part of the property.

It is possible to not prove the property but also have no counter examples. This can happen in cases where the antecedent does not provide enough information to prove or disprove the property. This is called a weak disagreement and requires changing the indexing relation to obtain a proof/counter example.

Correctness of Indexing Transformation and Output Constraint Checking

We prove that the above procedure for indexing transformation and output constraint checking is correct.

Here we only need to focus on proving the correctness of positive output constraint checking, i.e. that under a certain condition, a signal will be high. The proof for the negative output constraint checking is analagous.

In fact, with our formulation of relational properties later on, we don't even need to deal with guards, but our proof of correctness will deal with the general case.

Symbolic Simulation Invariant

Consider some residual $H[X, C]$ (the highexpr in a symbolic simulation evaluation sequence) in a signal $s(C, T)$ at a fixed time step. We always have the fact that

$$H[X, C] \wedge R[X, C, T] \rightarrow s(C, T)$$

(For all X, C, T)

This can be proved via induction on the fan-in of S .

The basecase is where s is an input or state variable corresponding to a certain time step, with $\text{highexpr } P$. In this case, since we apply the strong preimage to the antecedents, we have that

$$H = P^R = \text{dom}(R) \wedge \forall T (R[X, C, T] \rightarrow P[C, T])$$

This directly gives us that for all X, C, T where $H[X, C] \wedge R[X, C, T]$, we have $P[C, T]$. But $s(C, T) = P[C, T]$ so we are done.¹

An inductive case example:

Suppose $s = s_1 \wedge s_2$. We have Q_1 and Q_2 as the residuals of s_1 and s_2 respectively.

- $Q_1[X, C] \wedge R[X, C, T] \rightarrow s_1(C, T)$
- $Q_2[X, C] \wedge R[X, C, T] \rightarrow s_2(C, T)$
- So $Q_1[X, C] \wedge Q_2[X, C] \wedge R[X, C, T] \rightarrow s_1 \wedge s_2$
- But is equivalent to $H[X, C] \wedge R[X, C, T] \rightarrow s$ since $Q = Q_1 \wedge Q_2$ via the symbolic simulator

Output Constraint Checking

Output constraints that we wish to prove are of the form:

$$\forall C \forall T (P[T, C] \rightarrow s(C, T))$$

If we have that $P_R[X, C] \rightarrow H[X, C]$ and the coverage condition $\forall T \forall C (B[C, T] \rightarrow \exists X (R[X, C, T]))$ holds, then this will be true.

First remember that $P_R = \exists T (R[X, C, T] \wedge P[C, T])$.

Fix some C, T such that $P[C, T]$ is true.

- Since $P[C, T]$ is true, then $\forall X (R[X, C, T] \rightarrow P_R[X, C])$ by the definition of P_R .
- Since $P[C, T]$ is true, so is $B[C, T]$
- Consider some X^* such that $R[X^*, C, T]$ is true, this must exist by the coverage condition
- Since $P[T, C]$ and $R[X^*, C, T]$ are true, then we must have $P_R[X^*, C]$
- But now since $P_R[X, C] \rightarrow H[X, C]$, we have $H[X^*, C]$ as well
- Using the symbolic simulation invariant, since $H[X^*, C]$ and $R[X^*, C, T]$ are true, then $s(C, T)$ is true
- Since this did not rely on the values of C, T , then we have that $\forall C \forall T (P[T, C] \rightarrow s(C, T))$ is true

Negative Output Constraints

We can set up an analogous invariant for the lowexpr values in the symbolic simulation. We will then be able to prove that if L is the lowexpr of $s(C, T)$, then $(L[X, C] \wedge R[X, C, T] \rightarrow \neg s(C, T))$.

We can then prove output constraints of the form $\forall C \forall T (P[T, C] \rightarrow \neg s(C, T))$ in a similar manner to the highexpr case.

For our purposes where we use just need to prove that a certain wire is always high, we don't need this component. That being said, analysis of $L[X, C]$ is important for finding weak disagreements and counter examples.

Counter Example Analysis

Suppose that we have $P^R \rightarrow L \neq \text{False}$, let (X, C) be such that $(P^R \rightarrow L)[X, C]$ is true.

¹Note that we don't use the domain part. Indeed, the analysis will still work, but the weak preimage serves to remove useless/inconsistent indexing cases. This will help in avoiding false counter examples.

Converting Relational Constraints to Functional Constraints

Traditionally STE only deals with functional properties. However, we can employ a trick to also check relational properties.

Our relational properties are initially written as SytemVerilog assertions. We observe that each SV assertion can have its logic be shifted out of the property and into the surrounding specification circuit.

This means that we only need to check properties that are of the form `##k signal_name` where k is the number of clock cycles required to establish a certain property and `signal_name` is the name of the signal that corresponds to the property being true. We will call this signal, the “**property wire**” as its truth value reflects the truth value of the property.

Properties of this form are actually functional properties and thus can be written in the above 4 tuple form easily as `cout = [(signal_name, k:k, TRUE, FALSE)]`. We can prove this for k and then use the time shift to prove the rest of the ticks $> k$.

Indexing Transformations for Relational Properties

An additional benefit from this encoding method is that the property guards are extremely simple and thus have some nice properties related to the indexing transformation.

To check the properties, we would usually apply the weak preimage image operation on the guards. However, we have the following:

- $\text{True}_R = \text{True}^R = \text{dom}(R)[X, C]$
- $\text{False}_R = \text{False}^R = \text{False}$

This means that we only need to take 1 single preimage operation to get the domain and use that for checking all the various properties written in this form. In fact, computing the domain of an indexing relation generated from our automatic abstraction algorithm is a very simple operation that will be described later.

Furthermore, analysis of counter examples is also simplified. Simply, $\text{True}^R \rightarrow L = L$ so any time L is not false, we have found a counter example. Specifically, any T, C in the image of L is a counter example.

Checking Properties under Environmental Constraints